

Project 3: Active Learning for Learning-to-Defer

Human-Centric Artificial Intelligence

Experimental Report

1 Task 1: Baseline Classifier

1.1 Dataset

The project uses the AG News dataset for topic classification. Each news article belongs to one of four categories: *World*, *Sports*, *Business*, or *Sci/Tech*. The training set is used to train the classifier, while the test set is kept separate for evaluation, as required by the project specification.

1.2 Text Representation: TF-IDF

Since the input consists of news articles in natural language, the text first needs to be converted into numerical features. We selected Term Frequency–Inverse Document Frequency (TF-IDF) for this purpose.

TF-IDF assigns higher weights to words that are important for a particular article but occur less frequently across the complete dataset. This is useful for AG News because topic-specific words can provide strong signals for classification. For example, terms related to matches and teams are useful for identifying Sports articles, while terms related to companies and markets can indicate Business articles.

TF-IDF was selected instead of using only word counts because simple word frequencies give similar importance to common words that may occur across many categories. More advanced alternatives, such as word embeddings or transformer-based representations, can capture richer semantic and contextual information, but they require more computational resources and introduce additional complexity. For this project, TF-IDF provides a simple, efficient, and interpretable representation that is well suited to a strong baseline text classifier.

1.3 Classifier: Logistic Regression

For classification, we selected Logistic Regression. Although the name suggests binary classification, Logistic Regression can also perform multi-class classification, which makes it suitable for the four AG News categories.

The main reason for choosing Logistic Regression is its effectiveness with high-dimensional and sparse feature spaces such as TF-IDF. It is computationally efficient and provides a strong baseline without requiring a complex neural network architecture. It also produces confidence-related information that can be useful for later decisions about whether the classifier should make a prediction or defer to an expert.

Other models such as Naive Bayes, Support Vector Machines, decision trees, random forests, and neural networks could also be applied. Naive Bayes is a strong and simple text-classification method, while Support Vector Machines can provide competitive performance on sparse text data. However, Logistic Regression offers a good balance between accuracy, simplicity, efficiency, and interpretability for the current project. More complex transformer-based models could potentially achieve higher accuracy, but would add unnecessary complexity for establishing the baseline required before introducing the simulated expert.

1.4 Classifier Results

The Logistic Regression classifier was trained using the available labeled training data and evaluated on the complete test dataset. The overall test accuracy achieved was **91.59%**.

Table 1: Classifier performance on the AG News test set

Category	Accuracy
World	90.37%
Sports	98.32%
Business	88.00%
Sci/Tech	89.68%
Overall	91.59%

The classifier performs best on Sports, with an accuracy of 98.32%. This indicates that Sports articles contain relatively distinctive vocabulary that can be captured effectively using TF-IDF. Business has the lowest accuracy at 88.00%, suggesting that some Business articles are more difficult to distinguish from the other categories.

Overall, the result of 91.59% provides a strong baseline for the project and serves as the performance level against which the expert and later human-AI collaboration can be evaluated.

2 Task 2: Simulated Expert

2.1 Expert Design

The second task requires an imperfect simulated expert that demonstrates expertise in specific regions of the input space rather than being equally accurate

everywhere. To satisfy this requirement, the simulated expert was designed with different levels of competence across the four AG News categories.

The expert is intentionally strong in *Sports* and *Business*, while being weaker in *World* and *Sci/Tech*. For Sports and Business articles, the expert has a high probability of producing the correct label. For World and Sci/Tech articles, the probability of producing the correct label is substantially lower. When the expert does not produce the correct label, another category is selected.

This design creates a non-perfect expert with a clear competence profile. Consequently, the expert is useful for some types of news but less reliable for others, which is important for the project’s human-AI collaboration setting.

2.2 Expert Performance

The simulated expert was evaluated on the same complete test dataset. Its overall accuracy was **59.53%**. The category-level results are shown in Table 2.

Table 2: Classifier and simulated expert performance

Category	Classifier	Expert
World	90.37%	29.95%
Sports	98.32%	91.21%
Business	88.00%	89.74%
Sci/Tech	89.68%	27.21%
Overall	91.59%	59.53%

The results confirm the intended strengths and weaknesses of the simulated expert. The expert performs strongly on Sports (91.21%) and Business (89.74%), while its performance on World (29.95%) and Sci/Tech (27.21%) is considerably lower.

This difference is important because the expert is not simply a second classifier with random errors. Instead, it has a specific competence profile. The classifier is considerably stronger overall, but the expert performs competitively in Business and relatively well in Sports. This provides the required setting in which an intelligent system can later determine which predictions should be handled by the classifier and which may benefit from expert assistance.

3 Task 3: Learning to Defer

3.1 Approach

In this task both the classifier and the expert are available, so the system needs to decide, for each input, whether to trust its own prediction or query the expert instead. The main design question is how this decision should be made, since at prediction time the true label is obviously not known and the decision has to rely only on information that is available before the fact.

We approached this as a supervised learning problem on top of the existing classifier. The idea is to train a second, lightweight model, referred to here as the rejector, whose only job is to predict whether deferring to the expert would be beneficial for a given input. To generate training data for the rejector without leaking information, we used five fold cross validation to obtain out of fold predictions from the classifier on the training set. This means that for every training example we have a prediction that was made by a model that did not see that example during training, which mirrors how the classifier will behave on genuinely new data.

For every training example we then simulated the expert’s answer and compared both the classifier’s out of fold prediction and the expert’s answer to the true label. An example was labeled as should defer only if the expert would have been correct and the classifier would have been wrong, since these are exactly the cases where deferring is strictly better. The rejector was trained on this label using three simple, label free features: the classifier’s confidence (its highest predicted probability), the margin between the top two predicted probabilities, and the classifier’s predicted class itself. The predicted class turned out to be an important feature because the expert’s competence, as designed in Task 2, is strongly tied to the underlying topic rather than being uniform across classes.

A first version of this approach used a single global probability threshold to decide when to defer, but this performed worse than simply never deferring. Because the expert’s competence is so different across categories (strong on Sports and Business, weak on World and Sci/Tech), a single threshold is not expressive enough: it either defers too often on categories where the expert is unreliable, or too rarely on the one category where the expert is actually competitive with the classifier. We resolved this by tuning a separate deferral threshold for each of the four predicted classes, using a held out validation split of the training data, chosen to directly maximize final system accuracy rather than to match the should defer label itself.

3.2 Results

Table 3 summarizes the performance of the resulting learning to defer system on the complete test set, compared to the classifier alone and the expert alone.

Table 3: Learning-to-defer system performance	
System	Accuracy
Classifier only	91.59%
Expert only	59.53%
Learning-to-defer system	91.71%
Oracle upper bound (perfect deferral)	96.01%

The learning to defer system reaches 91.71%, a small but consistent improvement over the classifier alone. The overall deferral rate is 7.17% of test examples. The oracle upper bound of 96.01%, which represents the accuracy

that would be achieved if the system always chose whichever of the classifier or the expert happened to be correct, shows that there is still a meaningful gap between our learned policy and what a perfectly informed deferral decision could achieve. This gap is expected, since the rejector only has access to the classifier’s own confidence signals and not the true label.

Table 4 breaks the results down by category. As anticipated from Task 2, deferral is concentrated almost entirely on Business, where the expert’s accuracy (89.74%) is close to the classifier’s (88.00%), so sending uncertain classifier predictions to the expert has a genuine chance of correcting them. For the other three categories the deferral rate stays low, since the expert is either clearly weaker (World, Sci/Tech) or clearly not needed (Sports, where the classifier is already extremely strong).

Table 4: Per-class deferral rate and system accuracy

Category	Deferral Rate	System Accuracy
World	3.05%	91.11%
Sports	3.42%	98.37%
Business	15.74%	87.47%
Sci/Tech	6.47%	89.89%

Beyond raw accuracy, the task also asks us to reflect on the quality of the deferral decisions themselves, not only the resulting classification accuracy. One useful way to think about this is that the deferral policy needs to be evaluated against a moving target: whether deferring helps for a given example depends on a single, noisy draw of the simulated expert’s answer, so an example that would have benefited from deferral on one query might not on another. This inherent noise limits how sharply any deferral policy can be judged on a per example basis, and it is one reason the learned system, while consistently better than never deferring, does not close the full gap to the oracle bound.

4 Task 4: Active Learning for Expert Competence Discovery

4.1 Approach

The setting changes in this task: we no longer assume that expert labels are available during training. The classifier can still be trained on the full dataset as before, since its labels always exist, but the deferral policy from Task 3 can no longer be learned directly, because we do not know in advance where the expert is competent. Instead, we have to choose which training examples to actually query the expert on, with the goal of learning a good deferral policy using as few queries as possible.

Our first idea was to use uncertainty sampling, querying the expert only on the examples where the classifier’s own confidence is lowest. The reasoning is

that Task 3 already showed the deferral relevant region is concentrated where the classifier is uncertain, and the classifier’s confidence is available for every example without needing to query the expert, since the classifier is already trained on the full dataset.

When we tested this directly, however, it performed poorly at small query budgets, with system accuracy dropping close to the expert’s own accuracy. Investigating the cause, we found that querying only low confidence examples produces a training pool for the rejector that is systematically unrepresentative of the overall confidence distribution the classifier will actually encounter at test time. As a result, the rejector trained on this biased pool ends up deferring on the large majority of examples, including the many high confidence, high accuracy predictions that make up most of the test set, which is exactly the opposite of what a good policy should do.

To fix this, we used a hybrid querying strategy that interleaves uncertainty based queries with randomly selected queries in equal proportion. The random half keeps the labeled pool representative of the full confidence distribution, which is needed so that the rejector’s decision threshold is properly calibrated, while the uncertainty based half still concentrates queries in the region that matters most for the deferral decision. We compared this hybrid strategy against pure uncertainty sampling and pure random sampling across a range of query budgets, from 200 up to 40000 out of the 120000 available training examples.

4.2 Results

The three strategies behave very differently at small query budgets. Pure uncertainty sampling collapses, reaching only around 59% system accuracy, essentially defaulting to always deferring, until the query budget becomes large enough for the sampled pool to contain a broader mix of examples. Random sampling performs close to the classifier only baseline from very small budgets onward, largely because with few labeled examples the rejector has little confident signal and effectively defaults to deferring rarely, which happens to be close to optimal on this dataset since the overall benefit of deferral is modest to begin with. The hybrid strategy avoids the collapse seen in pure uncertainty sampling while still being able to concentrate its queries usefully once the budget grows.

By around 5000 to 10000 queries, all three strategies converge to a similar system accuracy, close to the fully supervised result from Task 3 (91.71%). This means the deferral policy can be learned using well under ten percent of the available training examples without a meaningful loss in system accuracy, which was the goal of this task.

We think the most informative result from this task is not simply which strategy reaches the highest accuracy, but the failure mode of naive uncertainty sampling itself. It illustrates a general point about active learning for a task like deferral: sampling strategies that are well justified for improving a classifier are not automatically appropriate for calibrating a decision threshold, because the threshold has to remain accurate over the full input distribution, not just over the region the sampling strategy focuses on. The hybrid strategy addresses

this by explicitly preserving a representative sample alongside the informative one, and we would recommend this combination whenever the goal is to learn a decision boundary rather than only to improve raw predictive accuracy.